

Trajectory-Level Redirection Attacks on Vision-Language-Action Models

Anonymous Author(s)

Affiliation

Address

email

Abstract: Vision-language-action (VLA) policies bring natural language into closed-loop robot control, enabling robots to execute manipulation tasks directly from text instructions. The same interface gives text a recurring role in control because the prompt is reused at every replanning step, and each prompt-conditioned action changes the future observations on which the policy acts. Existing VLA attacks study adversarial prompts that elicit targeted low-level actions or make such actions persist across changing images. We identify a stronger trajectory-level failure mode: a prompt that still *appears* to specify the intended task but redirects the final physical outcome. We mathematically formalize this setting as *command-preserving trajectory redirection*, a prompt-only threat model in which the attacker chooses one prompt before the episode, all policy and environment components remain fixed, and the prompt must stay close to the benign instruction while omitting target words and correction language. To find such prompts, we introduce an on-policy prompt search method that uses rollouts to discover perturbations whose closed-loop behavior tracks a target task while satisfying the command-preserving constraints. Experiments in simulation and on hardware show that near-benign prompt perturbations can redirect VLA rollouts to attacker-specified targets. These results expose a trajectory-level vulnerability in VLA instruction grounding: text that appears to preserve the intended command can still give an adversary control over the robot’s final physical outcome. [Project website:](#)

Keywords: Vision-Language-Action Models, Adversarial Attacks, Robot Safety

1 Introduction

Vision-language-action (VLA) policies are making natural language a persistent interface for closed-loop robot control. Models such as $\pi_{0.5}$ [1], OpenVLA [2], and RT 2 [3] map a natural-language instruction and a camera observation directly to robot actions. This interface is powerful: it enables non-expert users to specify manipulation tasks in natural language and allows a single policy to generalize across diverse scenes and objects. At the same time, it changes the role of language in the control loop. The instruction is not consumed once; it is reused at every replanning step, where it is conditioned on the current observation and influences the next action. As VLA-controlled robots move from constrained demonstrations toward broader deployment [1, 4], understanding the failure modes introduced by this persistent language interface becomes a concrete safety problem.

Recent work has begun to reveal these vulnerabilities. Jones et al. [5] showed that adversarial textual suffixes, optimized with the GCG algorithm [6], can drive OpenVLA to targeted low-level actions and even sustain them across many rollout steps, and VLA-Fool [7] broadened the attack surface to visual noise, adversarial patches, and cross-modal misalignment. These studies establish that VLA policies are vulnerable to input perturbations at the level of individual policy queries.

37 However, eliciting a targeted action at one inference step, or making that action persist across chang-
 38 ing images, is not equivalent to controlling what the robot *physically accomplishes*. A manipulation
 39 task such as “put the bowl on the plate” requires a temporally structured sequence of distinct be-
 40 haviors: reaching, grasping, lifting, transporting, aligning, and releasing, while the observations
 41 encountered later in the rollout are themselves determined by the prompt-conditioned actions taken
 42 earlier. Thus, a prompt optimized on a fixed set of pre-collected observations is evaluated on the
 43 wrong state distribution: the relevant observations are precisely those induced by the candidate
 44 prompt in closed loop. This motivates our central question: *can a command-preserving text per-*
 45 *turbation redirect the full closed-loop trajectory of a frozen VLA toward an adversary specified,*
 46 *alternate physical target?* By *command-preserving*, we require that the prompt remain textually
 47 close to the benign instruction, omit the attacker’s target task, and contain no override or correction
 48 language, thereby ruling out trivial prompt replacement. To find such perturbations, we introduce an
 49 on-policy prompt search algorithm that rolls out candidate prompts, relabels their visited observa-
 50 tions with actions from the same frozen VLA queried under the target instruction, and re-optimizes
 51 under the command-preserving constraints. This procedure is the prompt-search analogue of DAG-
 52 ger [8]: it aggregates data from the state distribution induced by the current candidate rather than
 53 from an offline trajectory that becomes stale once the prompt changes the robot’s behavior. We
 54 further show that, under standard smoothness assumptions, matching the target teacher along the
 55 perturbed prompt’s own trajectory controls terminal deviation from the target trajectory, whereas
 56 optimization on benign-rollout observations alone provides no such guarantee.

57 We evaluate our attack across multiple VLA architectures on LIBERO [9], spanning discrete-token
 58 action prediction, flow-matching and diffusion action heads, continuous action chunks, action-as-
 59 text designs, and a non-VLA diffusion-policy baseline. The results show that near-benign textual
 60 perturbations can reliably redirect closed-loop rollouts across diverse model families, with on-policy
 61 aggregation playing a critical role.

Our contributions: (i) We mathematically formalize *command-preserving trajectory redirection*: a prompt-only threat model in which the attacker changes a small part of the instruction, keeps the prompt apparently aligned with the benign command, and nevertheless redirects the robot to an adversary-specified alternate physical target. (ii) We introduce an on-policy prompt search method to find such command preserving trajectory redirection prompts. (iii) Empirically, we show that command-preserving perturbations expose a broad vulnerability across robot policy families, achieving attack success rates above 90% on seven of the nine evaluated architectures. (iv) We further validate the attack on physical hardware after embodiment-specific finetuning. (v) Finally, we evaluate defenses to such trajectory-level attacks, showing that robust mitigation requires command-level normalization rather than only surface cleanup.

63 2 Related Work

64 *Foundation Models for Robotics:* Vision Language Action Models (VLAs) unify visual perception,
 65 language grounding, and low-level motor control in a single prompt-conditioned policy. Recent
 66 systems span autoregressive action-token prediction [10, 3, 11, 12, 2, 13], discretized or vector-
 67 quantized action tokenizers [14, 15, 16], continuous action heads [17], diffusion or flow-based ac-
 68 tion generation [18, 4, 19, 20], action chunking [21], spatially enriched representations [22, 23],
 69 compact/efficient variants [24, 25, 26], and direct action-as-text decoding [27, 28]. Across these
 70 architectures, the language input directly shapes the action distribution queried repeatedly during
 71 closed-loop execution. Our work targets this shared interface. We ask whether small perturbations
 72 that remain confusable with the original command can redirect the resulting physical trajectory
 73 across different open VLA designs and action-decoding mechanisms.

74 *Adversarial attacks on language models and VLAs:* Adversarial text attacks originated in language
 75 models, including gradient-guided token or character edits such as HotFlip [29] and DeepWord-
 76 Bug [30], input-agnostic universal triggers [31], and Greedy Coordinate Gradient (GCG) suffix
 77 optimization for jailbreaking aligned LLMs [6]. Our prompt search builds on this discrete text-
 78 optimization toolkit but adapts it to embodied closed-loop control, where the adversarial text must

79 redirect a physical trajectory rather than a single textual output. In the VLA setting, Jones et al. [5]
 80 adapt GCG [6] to OpenVLA by optimizing a suffix that elicits a fixed target action, sustaining it for
 81 up to $28\times$ more rollout steps than a nominal instruction, while VLA-Fool [7] studies visual noise,
 82 adversarial patches, and cross-modal misalignment, including a semantically guided GCG variant
 83 on LIBERO [9]. However, these attacks primarily optimize and evaluate repetition of a single ac-
 84 tion under burn-in or nominal-trajectory observations, whereas our attack exposes a complemen-
 85 tary trajectory-level vulnerability: the prompt induces no individually unsafe action, yet redirects
 86 a closed-loop sequence of plausible actions toward an unintended physical target because future
 87 observations depend on earlier attacked actions.

88 3 Threat Model and Problem Statement

89 A VLA uses language as a persistent condition-
 90 ing signal for feedback control. The task text is
 91 embedded with the current observation at every
 92 policy query, and the decoded action changes
 93 the state from which the next observation is col-
 94 lected. Figure 1 illustrates one instance of the
 95 threat model. Throughout the section, the defi-
 96 nitions are stated for a general episode e with
 97 a benign task and an attacker-specified target
 98 predicate; the bowl–stove–plate setup is used only as the running example. The attack surface is
 99 only the task text; the robot, physical environment, environment dynamics, and policy are fixed.

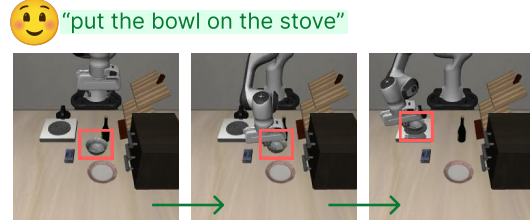


Figure 1: Running example. *add more*

100 **Closed-Loop VLA Rollouts.** Let $\Pi_\theta : \mathcal{T} \times \mathcal{O} \rightarrow \mathcal{A}^H$ denote a frozen VLA policy. The input
 101 $\tau \in \mathcal{T}$ is a text instruction, $o \in \mathcal{O}$ is an observation, and $\Pi_\theta(\tau, o)$ is an action chunk of horizon H .
 102 This abstraction covers both token-decoding VLAs and continuous-action VLAs after the model-
 103 specific action decoding step. At replanning step k , the robot observes $o_k = g(s_k)$, queries the
 104 VLA, and executes the first $m \leq H$ actions before the next query. Let $U_m : \mathcal{A}^H \rightarrow \mathcal{A}^m$ denote the
 105 executed-prefix operator. For episode e with initial state $s_0(e)$, the rollout induced by prompt τ is

$$s_{k+1}^\tau = F_m(s_k^\tau, U_m(\Pi_\theta(\tau, g(s_k^\tau)))), \quad s_0^\tau = s_0(e). \quad (1)$$

106 We write $\xi_e^\tau = (s_0^\tau, o_0^\tau, A_0^\tau, \dots, s_K^\tau)$ for the resulting trajectory where K is the number of replanning
 107 steps in the episode. The same prompt τ is used at every replanning step $[\cdot]$, while the observations
 108 o_k^τ are produced by the states reached under that prompt. Thus, the text perturbation changes both
 109 the action selected at a state and the future states on which later actions are selected $[\cdot]$.

110 **Adversary Capabilities.** The adversary controls the text instruction supplied before the episode
 111 starts. The policy weights, tokenizer, action decoder, robot embodiment, environment dynamics,
 112 initial state, observation process, action execution, and benchmark success function are fixed. The
 113 deployed attack is a single prompt τ used for the full rollout. For attack construction, the adversary
 114 may evaluate candidate prompts on the frozen VLA in the task environment. In particular, the adver-
 115 sary can roll out candidate prompts, record the observations they induce, and query the same frozen
 116 policy on those observations with different text prompts. Each episode e has a benign instruction τ_b
 117 that specifies the nominal task. The attacker specifies a scene-valid target task through a target set
 118 $\mathcal{Y}_e^{\text{tar}}$ over final physical states. In Fig. 1, τ_b is *put the bowl on the stove*, and the attacker target is
 119 the bowl-on-plate relation. Let $h : \mathcal{S} \rightarrow \mathcal{Y}$ map states to task-relevant relations. Let $\mathcal{Y}_e^{\text{bench}}$ be the
 120 benchmark success set. We define $B_e(\xi_e^\tau) = \mathbf{1}[h(s_K^\tau) \in \mathcal{Y}_e^{\text{bench}}]$ and $T_e(\xi_e^\tau) = \mathbf{1}[h(s_K^\tau) \in \mathcal{Y}_e^{\text{tar}}]$.

121 **Command-Preserving Prompt Perturbations.** The attack prompt must remain a valid perturbation
 122 of the benign command. The constraint is general: the perturbed prompt should still be read as the
 123 nominal task for episode e , while explicit text describing the attacker target must be absent. In Fig. 1,
 124 this means the prompt should still read as a version of *put the bowl on the stove*, while the attacker

task *put the bowl on the plate* must not appear in the prompt. The admissible family is

$$\mathcal{T}_{\text{cp}}(\tau_b, \Gamma_e) = \left\{ \tau \in \mathcal{T} : \underset{\substack{\text{small text change}}}{C_{\text{text}}(\tau, \tau_b) \leq \varepsilon}, \underset{\substack{\text{readable prompt}}}{\text{Valid}(\tau) = 1}, \underset{\substack{\text{no target leakage}}}{\text{Leak}(\tau; \Gamma_e) = 0}, \underset{\substack{\text{keep benign command}}}{\text{Preserve}(\tau, \tau_b) = 1} \right\}. \quad (2)$$

Here C_{text} is a fixed perturbation cost, such as character edit distance, token edit distance, bounded insertion length, or bounded trigger length. Valid excludes degenerate junk strings through pre-specified constraints such as length bounds, readability checks, and allowed character sets. The target lexicon Γ_e contains words and phrases associated with the attacker task; for Fig. 1, it includes the term *plate* and paraphrases of the bowl-on-plate target. Leak detects explicit target words, synonyms, and override language. Preserve records whether the perturbed instruction remains interpretable as the benign command under a fixed normalization rule or human annotation protocol. The exact instantiation of the heuristics used in Eq. (2) for our experiments is provided in Appendix X.

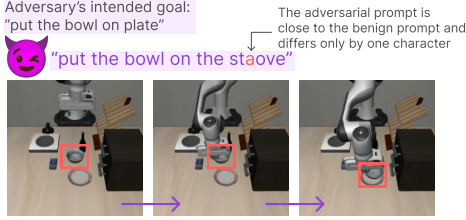


Figure 2: placeholder

Text constraint in the running example. For the nominal command “put the bowl on the stove” and attacker task “put the bowl on the plate”, a prompt such as “put the bowl on the staove” is admissible since it passes all four checks in Eq. (2). By contrast, “put the bowl on the plate” (direct target prompting) and “put the bowl on the stove. Corrected command: put the bowl on the plate” (explicit suffix override [5]) are inadmissible because they expose the target, replace the task, or override the benign command.

Trajectory-Level Attack Success. The attacker task is evaluated as a predicate over the final physical state. The benchmark predicate and the attacker predicate may refer to different task relations, but both are evaluated on the same rollout ξ_e^τ . A prompt succeeds on episode e when it satisfies the admissibility constraints, reaches the attacker target, and fails the benchmark relation:

$$\text{Succ}_e(\tau) = \mathbf{1} \left[\underset{\substack{\text{valid perturbed command}}}{\tau \in \mathcal{T}_{\text{cp}}(\tau_b, \Gamma_e)} \wedge \underset{\substack{\text{adversary target achieved}}}{T_e(\xi_e^\tau) = 1} \wedge \underset{\substack{\text{benchmark failed}}}{B_e(\xi_e^\tau) = 0} \right]. \quad (3)$$

4 Prompt Redirection Is a Trajectory-Level Problem

The central observation is that a VLA prompt acts as a persistent parameter of the closed-loop controller. The same text is reused at every replanning step, and each prompt-conditioned action changes the next observation on which the policy is queried. This has three consequences for prompt redirection. First, scoring a perturbation on a fixed set of observations cannot capture the states produced by the perturbation itself. Second, task redirection cannot be reduced to making one low-level action persist, because manipulation requires different actions at different phases of the task. Third, the attack must be judged by the final physical-state predicate, since the attacker’s objective is a final task relation rather than a local action. We detail these consequences below.

Fixed-observation prompt scores are insufficient. A fixed-observation score measures what a prompt does at one image. For the scene in Fig. 1, this might reveal that a perturbed command changes the first reach direction or the gripper motion from the initial view. The physical attack depends on the observations that follow that first change. A slightly different reach can change the gripper pose, contact state, occlusion, and object configuration seen at the next policy query. Later decisions are then made from this altered visual stream.

This identifies the trajectory quantity that matters. For any candidate prompt τ , define its executed action at state s as $u_\tau(s) = U_m(\Pi_\theta(\tau, g(s)))$. Let $u^* : \mathcal{S} \rightarrow \mathcal{A}^m$ denote any reference feedback controller whose rollout from $s_0(e)$ reaches the attacker target. The candidate’s on-trajectory mismatch to this reference behavior is

$$\epsilon_k^\tau = \|u_\tau(s_k^\tau) - u^*(s_k^\tau)\|. \quad (4)$$

The mismatch in Eq. (4) is evaluated at s_k^τ , the state produced by the candidate prompt itself. This is the state at which the prompt must make the next useful decision.

Formally, a score on a reference dataset such as $D_b = \{o_k^{\tau_b}\}_{k=0}^{K-1}$ evaluates the prompt on observations from the benign rollout. The attack objective depends on $o_k^\tau = g(s_k^\tau)$, where s_k^τ is generated by the candidate prompt through Eq. (1). The state distribution is part of what the prompt controls. This is the sequential distribution shift that motivates on-policy data aggregation in imitation learning [8]. The relevant observations are therefore the ones produced by the candidate prompt as it interacts with the task, including the intermediate states created by its own earlier actions.

Persistence is not (just) task redirection. Persistence captures whether an adversarial prompt continues to elicit a desired low-level action as visual inputs change [5]. The bowl-on-plate target in Fig. 1 requires a structured sequence of different actions: approach the bowl, grasp it, lift it, transport it to the plate, align above the plate, and release. The useful action before grasping is different from the useful action at release. A trajectory attack must therefore induce target-directed behavior as a function of the evolving scene. The attacker target is naturally expressed as a predicate over the final physical state. Many action sequences can realize the same target predicate, and many locally plausible action sequences can fail it. The relevant question is whether the perturbed instruction causes the closed-loop policy to move the system into $\mathcal{Y}_e^{\text{tar}}$ by the end of the rollout. We use a standard tracking bound to formalize this point. Let $s_{k+1}^* = F_m(s_k^*, u^*(s_k^*))$ be the reference rollout under u^* , with $s_0^* = s_0(e)$, and define $\delta_k = \|s_k^\tau - s_k^*\|$. Assume F_m is Lipschitz in state and action with constants L_s, L_u , so $\|F_m(s, u) - F_m(s', u')\| \leq L_s\|s - s'\| + L_u\|u - u'\|$, and assume u^* is L_* -Lipschitz. Let $\alpha = L_s + L_u L_*$. Then **Notation note**

$$\delta_K \leq L_u \sum_{k=0}^{K-1} \alpha^{K-1-k} \epsilon_k^\tau. \quad (5)$$

Thus, if the perturbed prompt chooses actions close to a target-achieving feedback behavior along the states it actually visits, its final physical state remains close to the corresponding target-achieving rollout. When $\alpha \geq 1$, early mismatches receive larger coefficients, matching the control intuition that early replanning errors shape the rest of the episode. The bound follows by applying the Lipschitz condition for F_m to the recursions for s_{k+1}^τ and s_{k+1}^* , then adding and subtracting $u^*(s_k^\tau)$ inside the action term. The first action difference is ϵ_k^τ , while the second is bounded by $L_* \delta_k$, giving $\delta_{k+1} \leq \alpha \delta_k + L_u \epsilon_k^\tau$. Unrolling from $\delta_0 = 0$ yields Eq. (5); the full proof is given in **Appendix X**.

Terminal physical state is the attack objective. Robot task benchmarks are judged by final physical-state predicates. In Fig. 1, the benchmark predicate is satisfied when the bowl ends on the stove. The attacker predicate is satisfied when the bowl ends on the plate (Fig. ??). The attack is successful only when the final physical state satisfies the attacker predicate and the benchmark predicate is unsatisfied. The tracking bound connects per-step behavior to this terminal predicate. Assume h is L_h -Lipschitz and the reference rollout reaches the attacker target with margin $\gamma > 0$, meaning the ball of radius γ around $h(s_K^*)$ lies inside $\mathcal{Y}_e^{\text{tar}}$. Then

$$L_h L_u \sum_{k=0}^{K-1} \alpha^{K-1-k} \epsilon_k^\tau \leq \gamma \implies T_e(\xi_e^\tau) = 1. \quad (6)$$

This condition states that sufficiently small on-trajectory action mismatch to a target-achieving behavior is enough to guarantee target success at the final physical state. Eq. (5) bounds $\|s_K^\tau - s_K^*\|$. Applying the Lipschitz property of h gives $\|h(s_K^\tau) - h(s_K^*)\| \leq L_h \|s_K^\tau - s_K^*\|$. The margin condition in Eq. (6) places $h(s_K^\tau)$ inside $\mathcal{Y}_e^{\text{tar}}$. Full proof in **Appendix X**.

The command-preserving constraint makes the failure mode precise. The text remains close to the benign task while the final physical state satisfies the attacker predicate. The vulnerability is the gap between apparent instruction grounding and the physical outcome produced by the controller.

5 On-Policy Teacher-Matching Prompt Search

We now describe an approach to search for a prompt $\tau \in \mathcal{T}_{\text{cp}}(\tau_b, \Gamma_e)$ that satisfies the trajectory-level success criterion in Eq. (3). The search uses the frozen VLA itself to provide action-level

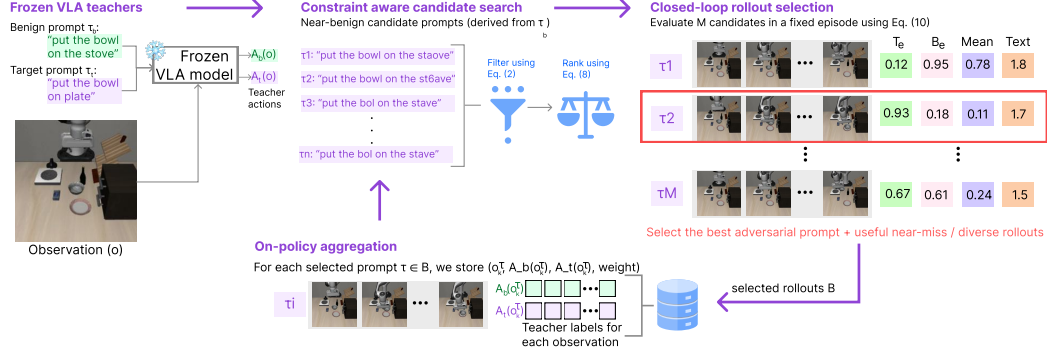


Figure 3: High level overview of the on-policy teacher-matching prompt search.

guidance: candidate perturbations are compared against the behavior induced by the benign task text and the attacker task text, then evaluated through closed-loop rollouts. A high level overview of the method is shown in Fig. 3. Implementation-specific details are available in Appendix X.

Teacher Labels from the Frozen VLA. Let τ_t denote a natural-language instruction for the attacker task. This prompt is used only during attack construction and is excluded from admissible attack prompts by the leakage constraint in Eq. (2). We first check target feasibility by verifying that τ_t reaches the attacker target, i.e., $T_e(\xi_e^{\tau_t}) = 1$. For any observation o , define the benign and target teacher chunks as $A_b(o) = \Pi_\theta(\tau_b, o)$ and $A_t(o) = \Pi_\theta(\tau_t, o)$. **Notation note** A candidate prompt τ produces $A_\tau(o) = \Pi_\theta(\tau, o)$. Since only the first m actions are executed before replanning, all comparisons use the executed-prefix loss $\ell_{\text{pre}}(A, A') = \sum_{j=1}^m \alpha_j \|A_j - A'_j\|_2^2$, $\alpha_j \geq 0$, $\sum_{j=1}^m \alpha_j = 1$. Let $\Delta(o) = \ell_{\text{pre}}(A_b(o), A_t(o)) + \eta$ with $\eta > 0$. We define normalized distances $d_t(\tau, o) = \ell_{\text{pre}}(A_\tau(o), A_t(o)) / \Delta(o)$ and $d_b(\tau, o) = \ell_{\text{pre}}(A_\tau(o), A_b(o)) / \Delta(o)$. Candidate prompts are scored by the target-vs-benign margin loss **change eqn number in fig3**

$$\ell_{\text{rank}}(\tau, o) = \max\{0, d_t(\tau, o) - d_b(\tau, o) + \mu\}, \quad (7)$$

where $\mu > 0$ is a margin. This favors prompts whose actions are closer to the target teacher than to the benign teacher at the same observation.

Constraint-Aware Candidate Search. At iteration i , the search maintains a labeled dataset $D_i = \{(o_r, A_b(o_r), A_t(o_r), w_r)\}_{r=1}^{n_i}$. Candidate prompts are generated using adversarial text perturbation and trigger operators from prior work [29, 30, 31, 6], together with mutations of high-scoring candidates from earlier iterations. Every candidate is filtered through Eq. (2).

For each surviving candidate, we compute

$$\hat{J}_i(\tau) = \frac{1}{\sum_r w_r} \sum_{(o_r, \cdot, w_r) \in D_i} w_r \ell_{\text{rank}}(\tau, o_r) + \frac{\lambda_t}{\sum_r w_r} \sum_{(o_r, \cdot, w_r) \in D_i} w_r d_t(\tau, o_r) + \beta C_{\text{text}}(\tau, \tau_b). \quad (8)$$

$\hookrightarrow$ target-vs-benign margin $\hookrightarrow$ target closeness $\hookrightarrow$ text cost

Here $\lambda_t, \beta \geq 0$ are fixed weights. The first term asks whether the candidate is more target-like, the second term favors closeness to the target teacher, and the final term favors smaller text perturbations. To avoid collapsing to one pattern, we select rollout candidates from multiple ranked lists: lowest $\hat{J}_i$, lowest d_t , largest target-vs-benign margin, and best prompt from each family.

On-Policy Aggregation and Rollout Selection. The initial dataset D_0 is built by rolling out τ_b and τ_t and labeling each visited observation with A_b and A_t . The benign rollout provides nominal task states, and the target rollout provides states along a feasible attacker-task trajectory. Subsequent iterations follow the dataset-aggregation principle from imitation learning [8]: prompts are evaluated on the states produced by the current search, and those states are added back into the scoring set.

At iteration i , the top M candidates under Eq. (8) are rolled out in the fixed episode. For each rollout, we record $T_e(\xi_e^\tau)$, $B_e(\xi_e^\tau)$, the text-validity checks, and the mean target distance on the candidate's own trajectory, $\bar{d}_t(\tau) = K^{-1} \sum_{k=0}^{K-1} d_t(\tau, o_k^\tau)$. Rollouts are ranked by

$$S_{\text{roll}}(\tau) = \lambda_{\text{tar}} T_e(\xi_e^\tau) - \lambda_{\text{bench}} B_e(\xi_e^\tau) - \lambda_{\text{dist}} \bar{d}_t(\tau) - \lambda_{\text{text}} C_{\text{text}}(\tau, \tau_b), \quad (9)$$

where all λ 's are nonnegative weights. For selected prompts $\mathcal{B}_i$, we add their observations to the dataset: $D_{i+1} = D_i \cup \{(o_k^\tau, A_b(o_k^\tau), A_t(o_k^\tau), w_k) : \tau \in \mathcal{B}_i, k = 0, \dots, K-1\}$. Weights w_k emphasize early replanning steps and observations where A_b and A_t differ. When a successful prompt is found, we greedily remove perturbation tokens and re-run the rollout, keeping the shortest prompt that remains in $\mathcal{T}_{cp}(\tau_b, \Gamma_e)$ and satisfies Eq. (3).

6 Experiments and Key Findings

Experimental Setup: In **simulation**, we evaluate on LIBERO [9] with the episode, initial state, environment dynamics, benchmark predicate, and policy fixed. We report the macro-average across LIBERO []-Spatial, Object, and Goal in the main paper. **Appendix X** reports the full suite-wise breakdown. For each scene, we choose a scene-valid attacker task using the simulator oracle state, then verify feasibility by rolling out the direct attacker-task prompt in the same fixed scene. We evaluate attacks only on pairs where the benign prompt succeeds on the benchmark task and the direct attacker-task prompt succeeds on the attacker task. **Hardware** experiments on a SO-100 6-DoF arm [] use the same prompt-only protocol, feasibility check, and metrics as simulation. All attacks use the success rule in Eq. (3). We test a wide variety of VLA families: discrete action-token VLAs [2, 13], generative and continuous action-head VLAs [4, 19, 18, 24, 32], optimized or custom VLA variants [17, 14] and action-as-text VLA variants [27]. All simulation results use publicly released LIBERO-finetuned variants of these models. For hardware, we fine-tune the corresponding model families on a small SO100 dataset; the dataset is available on the project website.

Metrics: Clean SR is benign-prompt benchmark success; Target-feasible SR is direct target-prompt success in the same fixed episode. Attack ASR is the percentage of attackable episodes where the search returns a valid command-preserving prompt whose rollout both fails the benchmark task and reaches the attacker target at the final state. Bench fail and Target final report those two rollout events separately. Edit is the median character-edit distance from the benign prompt, computed over Attack-ASR successes. Queries/success is the average number of policy queries per successful attack.

We report our Key Findings (KF) below:

KF#1: Near-benign prompts expose a shared trajectory-redirection vulnerability across VLA families. Across LIBERO, command-preserving perturbations redirect VLAs with different training recipes and action decoders (refer Table 1). The attacks not only make the robot fail, they also reliably move the rollout away from the benchmark task and toward the attacker's intended physical outcome (see Fig. 1, ??,6). This is the central trajectory-level result: across architectures with distinct training recipes and action representations, small local perturbations to a task instruction can redirect the full closed-loop behavior.

KF#2: Small perturbation budgets already suffice, while larger budgets mainly reduce search cost. Successful attacks do not require rewriting the instruction. Averaging the reported edit column over the VLA rows in Table 1 gives only 3.4 character edits per successful attack, excluding the non-VLA baseline. This shows that the vulnerable region is very close to the original command: the attacker does not need a long

Table 1: Command-preserving trajectory redirection across VLA families on LIBERO. Results are averaged over LIBERO episodes where both the benign task and the direct target task are feasible for the model.

Model	CSR (%)	TSR (%)	Attack ASR (%)	Bench fail (%)	Target final (%)	Edit
OpenVLA	76.5	69.4	91.8	94.6	93.2	3.7
MolmoAct	86.6	82.1	93.4	95.7	94.8	3.1
$\pi_{0.5}$	94.2	91.7	97.5	98.4	98.1	2.6
Octo	75.1	70.8	88.6	91.5	90.1	4.2
SmolVLA	88.8	85.4	94.7	96.1	95.3	3.3
GR00T-N1	93.9	92.6	96.8	98.0	97.6	2.5
OpenVLA	97.1	94.8	93.9	95.0	94.6	3.8
π_0 -FAST	85.5	82.9	95.6	96.9	96.2	2.4
VLA-0	94.7	91.9	82.8	84.4	83.7	5.4

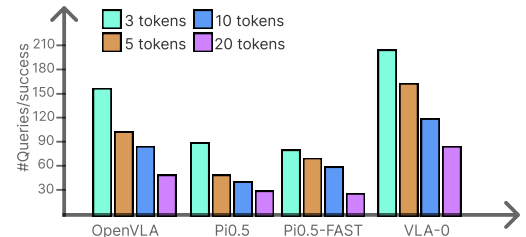


Figure 4: Perturbation budget versus search cost. We vary the maximum number of prompt tokens the search may modify and report the number of policy queries needed per successful attack.

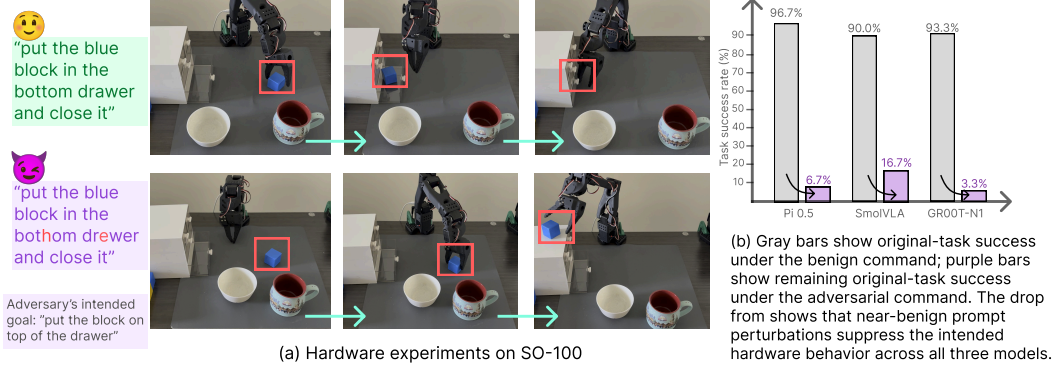


Figure 6: Hardware validation. (a) Qualitative results from SmolVLA. (b) Performance of SO100-finetuned VLAs.

suffix, a second instruction, or an explicit target phrase to redirect the rollout. The token-budget study in Fig. 4 shows the complementary tradeoff. With a tighter budget, the search has less freedom and needs more policy queries to find a successful perturbation; with a larger token budget, the same attack objective becomes easier to satisfy and the number of queries per success drops. Thus, smaller budgets give cleaner near-benign prompts while larger budgets make the search cheaper.

KF#3: The attack is carried by the corrupted destination representation. The causal trace shows that the prompt perturbation enters the policy through a specific part of the command: the corrupted destination phrase. In the example in Fig. 5, the adversarial destination tokenizes into pieces such as st, a, o, and ve. When we patch the residual-stream activations for these token-layer states back to their benign counterparts, the first action prefix loses its target-like behavior. The same intervention on ordinary command words such as put, bowl, or on has little effect. This localizes the attack to the internal representation of the destination, rather than to the whole prompt: a small corruption of the destination word creates a hidden state that steers the action head toward the alternate behavior.

KF#4: The attack survives real-robot deployment. The hardware results show that command-preserving redirection is not only a simulator artifact. Under the benign prompt, the models execute the original task reliably; under the adversarial near-benign prompt, the original task success collapses (see Fig. 6). The same class of small command perturbations also change real robot behavior after deployment-specific training, so the vulnerability persists through the full stack—language conditioning, visual perception, action decoding, and hardware execution.

KF#5: The attack remains target-like on the states it creates. The adversarial prompt does not only match the target teacher at the initial frame, it persists to behave like the target prompt along its own closed-loop rollout. To measure this, we first roll out the attack prompt, save the observations visited by the attacked policy, and then relabel each of those same observations with the benign prompt, the target prompt, and the attack prompt using paired flow

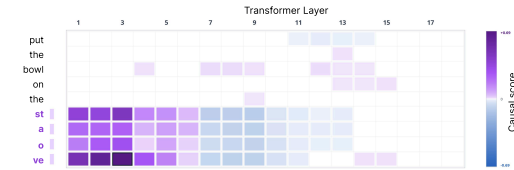


Figure 5: Causal trace of a command-preserving perturbation through $\pi_{0.5}$. For one fixed LIBERO observation, we patch each adversarial token-layer residual state to its benign counterpart and measure the change in the first action prefix. Purple indicate activations that support target-like action; blue indicate activations that oppose it; white cells have little effect. The heatmap is averaged over 10 different seeds.

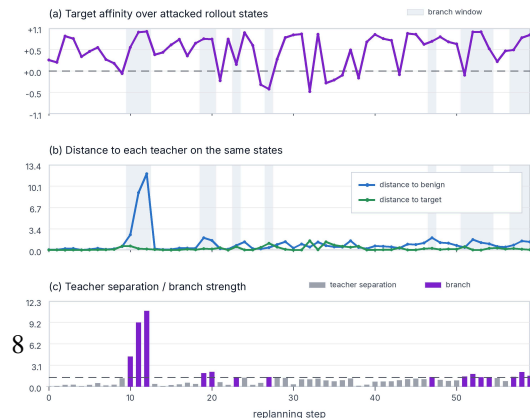


Figure 7: Closed-loop target affinity on attack-induced states. (a) shows target affinity over replanning steps; positive values mean the ad

noise. At each replanning step, we compare the attack action prefix to the benign-teacher and target-teacher action prefixes. Positive target affinity means that the attack action is closer to the target teacher than to the benign teacher. Fig. 7 shows that the attack stays target-like over most of the rollout, and especially during branch windows where the benign and target teachers disagree strongly—keeping the alternate behavior on the states produced by its own earlier actions, rather than only causing a one-step action change at the beginning.

KF#6: Defending against attacks requires

command-level normalization. The preprocessing results show a clear defense hierarchy (Table 2). Lightweight formatting changes such as whitespace normalization or Unicode normalization preserve clean behavior, but they leave much of the attack surface intact because many successful prompts remain valid near-benign commands after these transformations. Stronger transformations that repair or canonicalize the instruction sharply reduce attack success while still retaining most clean-task performance. Hence, the right defense is to place a command-normalization layer in front of the VLA: map noisy or corrupted instructions back to a small set of validated task commands before action generation.

Ablation takeaway. Table 3 shows that random valid perturbations rarely find a redirecting command, while fixed-observation scoring improves success but wastes queries because it does not track the states created by the attack. Using a target teacher helps, but the strongest search comes from comparing against both the benign and target teachers and then updating on the attacked rollout distribution, which finds successful command-preserving attacks more reliably and with far fewer policy queries.

Table 2: Effect of simple prompt preprocessing defenses. LIBERO-Goal $\pi_{0.5}$ model. [fill](#)

Preprocessing	Clean SR retained (%)	Attack ASR (%)	Prompts changed (%)
None	100.0	95.1	0.0
Whitespace normalization	99.4	83.7	41.8
Punctuation stripping	98.6	58.3	73.6
Unicode NFKC normalization	99.7	92.4	18.9
Spell correction	96.9	31.8	82.7
Nearest-task canonicalization	94.2	7.4	100.0

Table 3: Ablation of trajectory-level prompt search on LIBERO-Goal with $\pi_{0.5}$. The full method combines target-vs-benign teacher scoring with on-policy aggregation over attacked rollout states, yielding both higher attack success and lower search cost.

Method	Target teacher	Benign teacher	On-pol.	ASR (%)	Queries / succ.
Random perturbations	–	–	–	11.7	1846.3
Fixed-observation search	✓	✓	–	54.8	238.7
Target-teacher only	✓	–	✓	71.6	164.2
No on-policy aggregation	✓	✓	–	79.3	103.8
Full method	✓	✓	✓	95.1	41.6

References

- [1] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [3] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

- [5] E. K. Jones, A. Robey, A. Zou, Z. Ravichandran, G. J. Pappas, H. Hassani, M. Fredrikson, and J. Z. Kolter. Adversarial attacks on robotic vision language action models. *arXiv preprint arXiv:2506.03350*, 2025.
- [6] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.
- [7] Y. Yan, Y. Xie, Y. Zhang, L. Lyu, H. Wang, and Y. Jin. When alignment fails: Multimodal adversarial attacks on vision-language-action models. *arXiv preprint arXiv:2511.16203*, 2025.
- [8] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [11] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- [12] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [13] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.
- [14] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [15] Y. Wang, H. Zhu, M. Liu, J. Yang, H.-S. Fang, and T. He. VQ-VLA: Improving vision-language-action models via scaling vector-quantized action tokenizers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [16] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, et al. RoboCat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research (TMLR)*, 2023.
- [17] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [18] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [19] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

- 414 [20] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. RDT-1B: a diffusion
415 foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.
- 416 [21] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel, C. Garrett, F. Ramos, D. Fox, A. Li, A. Gupta,
417 et al. Hamster: Hierarchical action models for open-world robot manipulation. In *International*
418 *Conference on Learning Representations*, volume 2025, pages 24040–24068, 2025.
- 419 [22] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, et al.
420 Spatialvla: Exploring spatial representations for visual-language-action model. *arXiv preprint*
421 *arXiv:2501.15830*, 2025.
- 422 [23] Y. Ji, H. Tan, J. Shi, X. Hao, Y. Zhang, H. Zhang, P. Wang, M. Zhao, Y. Mu, P. An, et al.
423 Robobrain: A unified brain model for robotic manipulation from abstract to concrete. In
424 *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages
425 1724–1734, 2025.
- 426 [24] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi,
427 C. Pascal, M. Russi, A. Marafioti, et al. Smolvla: A vision-language-action model for afford-
428 able and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- 429 [25] T. Jiang, X. Jiang, Y. Ma, X. Wen, B. Li, K. Zhan, P. Jia, Y. Liu, S. Sun, and X. Lang. The
430 better you learn, the smarter you prune: Towards efficient vision-language-action models via
431 differentiable token pruning. *arXiv preprint arXiv:2509.12594*, 2025.
- 432 [26] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, et al.
433 Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation.
434 *IEEE Robotics and Automation Letters*, 2025.
- 435 [27] A. Goyal, H. Hadfield, X. Yang, V. Blukis, and F. Ramos. Vla-0: Building state-of-the-art vlars
436 with zero modification. *arXiv preprint arXiv:2510.13054*, 2025.
- 437 [28] D. Niu, Y. Sharma, G. Biamby, J. Quenum, Y. Bai, B. Shi, T. Darrell, and R. Herzig. Llarva:
438 Vision-action instruction tuning enhances robot learning. *arXiv preprint arXiv:2406.11815*,
439 2024.
- 440 [29] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou. Hotflip: White-box adversarial examples for text
441 classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational*
442 *Linguistics (ACL)*, 2018.
- 443 [30] J. Gao, J. Lanchantin, M. L. Soffa, and Y. Qi. Black-box generation of adversarial text se-
444 quences to evade deep learning classifiers. In *IEEE Security and Privacy Workshops (SPW)*,
445 2018.
- 446 [31] E. Wallace, S. Feng, N. Kandpal, M. Gardner, and S. Singh. Universal adversarial triggers for
447 attacking and analyzing nlp. In *Proceedings of the 2019 Conference on Empirical Methods in*
448 *Natural Language Processing (EMNLP)*, 2019.
- 449 [32] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox,
450 F. Hu, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv*
451 *preprint arXiv:2503.14734*, 2025.